

Consistência: Eventually vs Strongly Consistent Reads

Modelos de Consistência de Leitura no DynamoDB

AWS Certified Developer Associate

Instrutor: Marlon Anesi

Como DynamoDB Armazena Dados



Replicação automática

Cada item é replicado em 3 Availability Zones (AZs)

Garante durabilidade e alta disponibilidade

Write é confirmado após salvar em 2 AZs

Processo de escrita

1. Aplicação envia write para DynamoDB
2. DynamoDB grava na réplica primária
3. Replica para segunda AZ (síncrono)
4. Retorna sucesso para aplicação
5. Replica para terceira AZ (assíncrono em background)

Implicação para leituras

A terceira réplica pode estar TEMPORARIAMENTE desatualizada

Leitura pode retornar dados da terceira réplica (mais rápido, mas possivelmente desatualizado)

Ou pode ler das réplicas já sincronizadas (mais lento, mas sempre atualizado)

Eventually Consistent Reads



O que é

Leitura que PODE retornar dados desatualizados temporariamente
Lê de qualquer réplica (incluindo a que pode estar desatualizada)
Eventualmente (geralmente em milissegundos) fica consistente

Comportamento PADRÃO no DynamoDB

Se não especificar `ConsistentRead=true`, é eventually consistent

Características

- Mais rápido: pode ler de qualquer réplica
- Mais barato: consome metade dos RCUs
- Maior throughput disponível
- Pode retornar dados stale (< 1 segundo)

Quando usar

- Dados não críticos (dashboard, feeds)
- Aplicações que toleram pequeno lag
- Quando custo/performance importa mais que precisão imediata

Strongly Consistent Reads



O que é

Leitura que SEMPRE retorna dados mais recentes

Lê apenas de réplicas já sincronizadas

Reflete todos os writes anteriores

Como habilitar

Especificar `ConsistentRead=true` na chamada API

Funciona com `GetItem`, `Query`, `BatchGetItem`

Características

- Mais lento: espera sincronização
- Mais caro: consome DOBRO de RCUs
- Menor throughput disponível
- Garante dados sempre atualizados

Quando usar

- Dados críticos (saldo bancário, inventário)
- Read-after-write precisa ver última escrita
- Aplicações que não toleram dados stale

Eventually vs Strongly: Comparação

Aspecto	Eventually Consistent	Strongly Consistent
Padrão?	SIM (default)	NÃO (opt-in)
Latência	Menor	Maior
Custo (RCUs)	1 RCU	2 RCUs (dobro)
Dados atualizados?	PODE estar stale	SEMPRE atualizados
Throughput	Maior	Menor
GSI support	SIM	NÃO
Caso de uso	Feeds, dashboards	Saldos, inventário

Regra de decisão

- Use Eventually Consistent (padrão) para MAIORIA dos casos
- Use Strongly Consistent apenas quando dados stale causarem problemas
- Lembre: GSI SEMPRE é eventually consistent (não tem opção)

Pontos-chave para revisão

1. Eventually Consistent (padrão)

Pode retornar dados stale temporariamente

Mais rápido, mais barato (1 RCU), maior throughput

2. Strongly Consistent (opt-in)

SEMPRE retorna dados atualizados

Mais lento, mais caro (2 RCUs), menor throughput

3. Regra de decisão

Eventually: 90% dos casos (suficiente para maioria)

Strongly: dados críticos que não toleram stale

LEMBRE-SE: Eventually Consistent é PADRÃO. Strongly Consistent custa DOBRO de RCUs!